

MOTOR DE TEMPLATES

Handlebars

Como transformar dados do servidor em páginas HTML

Projeto app_escola · Express 5 · Node.js ESM

Desenvolvimento Web — Eixo Informação e Comunicação

E.E.E.P. Lúcia Helena Viana Ribeiro — Horizonte/CE

Prof. Francisco Ericson Cornélio da Costa



2

O problema: HTML preso dentro do JavaScript

Sem um motor de templates, a página vira uma colagem de strings.

SEM template engine

```
res.send(
  "<html><body>" +
  "<h1>" + titulo + "</h1>" +
  "<ul>" +
  turmas.map(t =>
    "<li>" + t.nome + "</li>"
  ).join("") +
  "</ul></body></html>"
);
```

COM Handlebars

```
// controller
res.render("home", {
  titulo, turmas
});

<!-- view -->
<h1>{{titulo}}</h1>
<ul>
  {{#each turmas}}
    <li>{{nome}}</li>
  {{/each}}
</ul>
```

Por que a primeira versão é ruim?

O designer não consegue mexer no HTML sem mexer no JavaScript. Uma aspa esquecida quebra a aplicação inteira. Não existe reaproveitamento: cabeçalho e rodapé são recopiados em cada rota. E qualquer texto vindo do usuário entra na página sem tratamento.

3

O que é um motor de templates

Um programa que junta um molde HTML com dados e devolve HTML pronto.

1. Template

Arquivo .handlebars com o HTML fixo e marcações indicando onde entram os dados.



2. Dados

Objeto JavaScript entregue pelo controller: títulos, listas, resultados do banco.



3. HTML final

Texto HTML completo, montado na hora e enviado ao navegador.

O navegador nunca vê o Handlebars.

Toda a montagem acontece no servidor. O que chega ao navegador é HTML comum, igual ao que vocês escrevem à mão. Se abrir o código-fonte da página no navegador, não haverá nenhuma chave dupla ali.

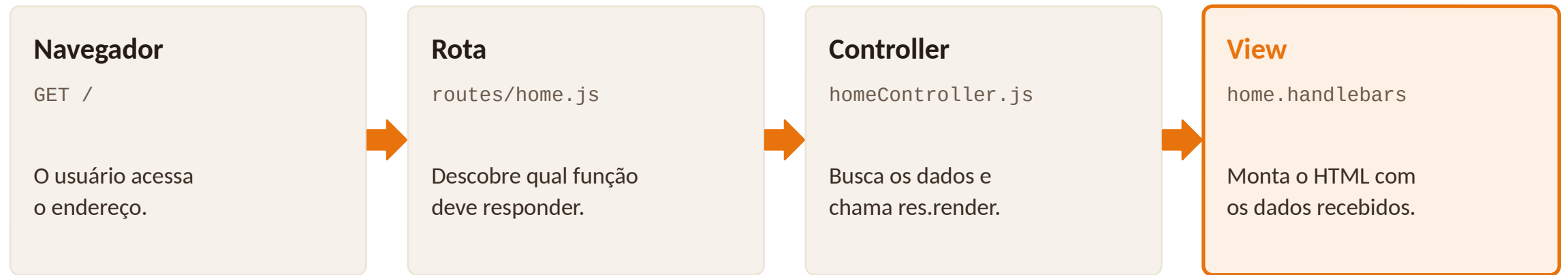
Analogia

É o mala direta do Word. O molde da carta é um só; a lista de nomes muda. O Handlebars é quem preenche o molde, uma vez para cada visita à página.

4

Onde o Handlebars entra no MVC

Ele é a camada View. Só cuida da aparência, nunca da regra de negócio.



O caminho de volta é sempre HTML puro, do servidor para o navegador.

A view PODE

Exibir valores, percorrer listas, mostrar ou esconder um trecho conforme uma condição simples.

A view NÃO PODE

Consultar o banco, calcular médias, validar dados ou decidir regras. Isso é trabalho do controller.

5

Instalação e configuração

Três passos: instalar o pacote, registrar o motor e apontar a pasta das views.

Terminal

```
npm install express-handlebars
```

src/app.js

```
import { engine } from "express-handlebars";

views() {
  this.app.engine("handlebars", engine({
    defaultLayout: "main",
    layoutsDir: resolve(__dirname,
      "views", "layouts"),
  }));
  this.app.set("view engine", "handlebars");
  this.app.set("views",
    resolve(__dirname, "views"));
}
```

app.engine

Ensina ao Express como processar arquivos com a extensão .handlebars.

defaultLayout

Define a moldura HTML aplicada a todas as páginas por padrão.

set("view engine")

Permite escrever res.render("home") sem repetir a extensão do arquivo.

set("views")

Informa em qual pasta o Express deve procurar os arquivos de template.

6

A pasta de views

Cada tipo de arquivo tem um lugar fixo. O Handlebars procura por convenção.

```
src/  
├── controllers/  
├── routes/  
└── views/  
    ├── layouts/  
    │   └── main.handlebars  
    ├── partials/  
    │   └── menu.handlebars  
    ├── home.handlebars  
    └── sobre.handlebars
```

Regra de ouro do nome

`res.render("home")` procura por `src/views/home.handlebars`. O nome no controller e o nome do arquivo precisam ser idênticos, inclusive em maiúsculas e minúsculas.

layouts/

A moldura da página: `<html>`, `<head>`, cabeçalho e rodapé. Normalmente um único arquivo, o `main`.

partials/

Pedaços reaproveitáveis: menu, card de aluno, formulário. Chamados de dentro de outras views.

views/ (raiz)

As páginas em si. Cada uma corresponde a um `res.render` do controller.

7 O layout e o {{{body}}}

A moldura fica em um arquivo só. Cada página preenche apenas o miolo.

views/layouts/main.handlebars

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <title>{{titulo}}</title>
  </head>
  <body>
    <header><h1>{{titulo}}</h1></header>

    <main>{{{body}}}</main>

    <footer>E.E.E.P. Lúcia Helena</footer>
  </body>
</html>
```

views/home.handlebars

```
<p>Bem-vindo ao sistema.</p>

<h2>Turmas</h2>
<ul>
  <li>Informática 1</li>
</ul>
```

{{{body}}} com três chaves

É o único lugar do sistema onde as três chaves são obrigatórias. O conteúdo da página já é HTML montado; com duas chaves as tags apareceriam escritas na tela.

Sem layout, cada página repetiria as 12 linhas da esquerda. Com layout, escrevemos uma vez só.

8

Do controller para a view

O segundo argumento de `res.render` é o único canal de dados entre as duas camadas.

controllers/homeController.js

```
class HomeController {  
  index(req, res) {  
    return res.render("home", {  
      titulo: "Sistema Escolar",  
      professor: "Ericson",  
      turmas: [  
        { nome: "Info 1", alunos: 42 },  
        { nome: "Info 2", alunos: 38 },  
      ],  
    });  
  }  
}
```

views/home.handlebars

```
<h2>{{titulo}}</h2>  
<p>Professor: {{professor}}</p>  
  
{{#each turmas}}  
  <p>{{nome}}: {{alunos}}</p>  
{{/each}}
```

Cada chave vira uma variável

As chaves do objeto (titulo, professor, turmas) são exatamente os nomes usados dentro das chaves duplas. Nome diferente, resultado vazio.

Variável que não existe não dá erro

O Handlebars simplesmente não escreve nada e a página carrega normal. Isso torna o erro de digitação difícil de perceber: confira o nome antes de suspeitar do código.



Expressões: as chaves duplas

A construção mais usada. Busca um valor no objeto de dados e escreve na página.

Escrevo na view	O que é	Aparece na página
<code>{{titulo}}</code>	valor simples	Sistema Escolar
<code>{{aluno.nome}}</code>	propriedade aninhada	Maria Souza
<code>{{turmas.length}}</code>	propriedade de array	2
<code>{{turmas.[0].nome}}</code>	índice numérico	Info 1

Índice pede colchetes

Escreva `{{turmas.[0].nome}}`. A forma `turmas.0.nome` não funciona no Handlebars.

Espaço não faz diferença

`{{titulo}}` e `{{ titulo }}` dão o mesmo resultado. Escolha um estilo e mantenha no projeto inteiro.

Lista sem #each

`{{turmas}}` sozinho imprime algo inútil, como `[object Object]`. Array sempre pede um bloco `#each`.

10

Duas chaves ou três? Uma questão de segurança

A diferença parece cosmética. Não é.

Dado recebido de um formulário:

```
comentario = "<script>roubarSenha()</script>"
```

{{comentario}} · duas chaves

O Handlebars converte os sinais < e > em código seguro. O navegador exibe o texto na tela e não executa nada.

Na tela aparece: `<script>roubarSenha()</script>`

{{{comentario}}} · três chaves

O conteúdo entra na página como código real. O navegador executa o script do atacante dentro do seu site.

Resultado: a função do invasor roda de verdade

Regra prática: use sempre duas chaves. As três chaves só no {{{body}}} do layout.

Esse tipo de ataque tem nome: XSS (Cross-Site Scripting). Está entre as falhas mais exploradas em aplicações web.

11

{{#each}} — percorrer listas

O bloco entre a abertura e o fechamento se repete uma vez para cada item do array.

views/turmas.handlebars

```
<table>
  {{#each turmas}}
    <tr>
      <td>{{@index}}</td>
      <td>{{nome}}</td>
      <td>{{alunos}} alunos</td>
    </tr>
  {{else}}
    <tr><td>Nenhuma turma cadastrada</td></tr>
  {{/each}}
</table>
```

Dentro do bloco, o contexto muda

{{nome}} passa a significar 'o nome do item atual', sem repetir turmas.

{{@index}}

A posição do item, começando em zero. Existe também {{@first}} e {{@last}}.

{{else}}

O que mostrar quando a lista chega vazia. Evita a temida tabela em branco.

Preciso do valor de fora do bloco?

Use duas barras e um ponto para subir um nível: {{../titulo}}. Isso busca o valor no contexto que envolve o each, e não no item atual.

{{#if}} e {{#unless}} — condicionais

Mostram ou escondem um trecho da página. Só isso: não fazem comparações.

Exibir conforme o dado

```
{{#if aprovado}}  
  <span class="ok">Aprovado</span>  
{{else}}  
  <span class="rec">Recuperação</span>  
{{/if}}  
  
{{#unless matriculado}}  
  <a href="/matricula">Faça sua matrícula</a>  
{{/unless}}
```

Não funciona:

```
{{#if nota >= 6}}      ·      {{#if idade == 18}}
```

O Handlebars não entende operadores. A comparação é feita no controller, que envia o resultado pronto: aprovado: nota >= 6.

O que conta como falso

false, 0, string vazia, null, undefined e array vazio. Todo o resto é verdadeiro.

unless é o if invertido

Executa quando a condição é falsa. Útil para avisos de ausência de dado.

Isso é proposital

A filosofia do Handlebars é manter a view sem lógica. Regra de negócio pertence ao controller.

Contexto: onde o Handlebars procura o valor

Cada bloco cria um novo ponto de partida para a busca dos nomes.

Dados enviados pelo controller

```
{
  escola: "Lúcia Helena",
  turma: {
    nome: "Info 1",
    alunos: [
      { nome: "Ana" },
      { nome: "Bruno" },
    ]
  }
}
```

Como alcançar cada valor

```
{{escola}}
{{turma.nome}}

{{#with turma}}
  {{nome}}
  {{#each alunos}}
    {{nome}} - {{../nome}}
  {{/each}}
{{/with}}
```

{{#with}}

Entra dentro de um objeto e evita repetir o caminho inteiro a cada linha.

this

Aponta para o item atual. Em uma lista de textos simples, {{this}} imprime o próprio texto.

../

Sobe um nível de contexto. Pode ser encadeado: ../../ sobe dois.

Partials — escrever uma vez, usar em todo lugar

Pedaços de HTML reaproveitáveis, como o menu ou o cartão de aluno.

1. Registrar a pasta em app.js

```
engine({
  defaultLayout: "main",
  layoutsDir: resolve(__dirname, "views", "layouts"),
  partialsDir: resolve(__dirname, "views", "partials"),
})
```

2. Criar views/partials/menu.handlebars

```
<nav>
  <a href="/">Início</a>
  <a href="/turmas">Turmas</a>
</nav>
```

3. Chamar de qualquer view

```
{{> menu}}

{{! passando dados }}
{{#each alunos}}
  {{> cardAluno aluno=this}}
{{/each}}
```

O sinal >

Significa 'inclua aqui o conteúdo daquele arquivo'. O nome vem sem a extensão.

Layout ou partial?

O layout envolve a página por fora e existe um por vez. O partial é chamado por dentro, quantas vezes você quiser, inclusive dentro de um each.

Helpers — quando o Handlebars não faz sozinho

Funções JavaScript registradas na configuração e chamadas de dentro da view.

src/app.js

```
engine({
  defaultLayout: "main",
  helpers: {
    moeda(valor) {
      return valor.toFixed(2).replace(".", ",");
    },
    maiorQue(a, b) {
      return a > b;
    },
  },
})
```

Sem vírgula entre argumentos

Escreve-se `{{moeda mensalidade}}`, separando por espaço. A sintaxe de chamada não é a do JavaScript.

Na view

```
R$ {{moeda mensalidade}}

{{#if (maiorQue nota 6)}}
  Aprovado
{{/if}}
```

Parênteses = subexpressão

Quando o resultado de um helper alimenta outro bloco, envolva a chamada em parênteses.

Use com moderação

Formatação e comparação, sim. Buscar dados ou aplicar regra de negócio, não.

O que não deve entrar na view

A pergunta de controle: se eu trocar o HTML inteiro, esta linha continua fazendo sentido?



Calcular a média das notas

O controller envia a média já calculada.



Consultar o banco de dados

A view recebe dados prontos, nunca vai buscá-los.



Decidir se o aluno foi aprovado

É regra de negócio: pertence ao controller ou ao model.



Exibir a média já calculada

Apresentar um valor pronto é o papel da view.



Percorrer a lista de alunos

Repetir um bloco com `{{#each}}` é apresentação.



Esconder um botão conforme uma flag

Um `{{#if}}` sobre um dado booleano já decidido.

Uma view difícil de ler quase sempre é sintoma de um controller preguiçoso.

Erros comuns e o que verificar

Quatro mensagens que vão aparecer no terminal de vocês nesta semana.

Failed to lookup view "home"

O Express não encontrou o arquivo.

Confira o nome em `res.render`, a extensão `.handlebars` e o caminho em `app.set("views")`.

The partial main could not be found

O layout não está onde deveria.

O arquivo precisa ser `views/layouts/main.handlebars` e `layoutsDir` precisa apontar para essa pasta.

A página abre, mas o dado some

Não é erro: é nome divergente.

A chave enviada pelo controller e o nome dentro das chaves duplas precisam ser idênticos.

As tags HTML aparecem escritas

Escapamento em ação.

Só use três chaves quando o conteúdo for HTML confiável, como o `{{{body}}}` do layout.

Leia sempre a primeira linha do erro e o caminho do arquivo citado nela.

Layout

Uma moldura, `{{{body}}}` no miolo.

Expressão

`{{nome}}` busca no objeto do render.

Segurança

Duas chaves sempre; três só no body.

#each

Repete o bloco e troca o contexto.

#if

Mostra ou esconde. Não compara.

Partial

`{{> menu}}` reaproveita um pedaço.

Atividade da aula

- No `app_escola`, crie a rota `/turmas` com controller, view e listagem usando `{{#each}}`.
- Mostre a mensagem “Nenhuma turma cadastrada” quando o array vier vazio, usando `{{else}}`.
- Crie o partial `menu.handlebars` e use-o nas páginas inicial e de turmas.

Dúvida durante a prática? Primeiro leia o erro. Depois confira o nome do arquivo. Só então me chame.